AUDITORIA TÉCNICA · CODE & ARCHITECTURE REVIEW

Simulados Policiais Pro

Varredura completa de arquitetura, segurança, camada de dados, resiliência e custos — com avaliação de prontidão para suportar 100.000 usuários ativos. Diagnóstico baseado em evidências de código, histórico do repositório e probes não destrutivos em produção.

Relatório de Auditoria de Engenharia

Z.ai · Principal Software Architect Review

Escopo: 20 rotas de API · camada de dados · frontend PWA · pipeline de IA · pagamentos
Método: inspeção somente-leitura · probes passivos · análise de 25 dimensões

23 DE OUTUBRO DE 2026 · PRODUÇÃO V0.2.1

Sumário

1. Resumo Executivo 1

2. Nota da Engenharia Atual 2

3. Arquitetura Atual 3

4. Diagrama do Sistema 4

5. Top 20 Problemas 5

6. Buracos Encontrados 6

7. Segurança 7

8. Multi-Tenancy e Isolamento 9

9. Banco de Dados e Camada de Dados 9

10. Escalabilidade (1.000, 10.000, 50.000, 100.000) 10

11. Performance 11

12. Resiliência 12

13. Observabilidade 12

14. Custos 13

15. O Que Quebra Primeiro 13

16. Roadmap de Escala 14

17. Arquitetura Recomendada 15

18. Overengineering a Evitar 16

19. Plano de Ação 16

20. Veredito Final 18

1. Resumo Executivo

> *Se eu fosse responsável por colocar 100.000 usuários neste sistema, minha avaliação seria: **não colocaria na forma atual.** A nota de prontidão é **3,2/10**. O produto funciona e a experiência local-first é um diferencial real de arquitetura, mas a camada de contas do servidor não existe como banco de dados, há sete riscos críticos confirmados com arquivo e linha — três deles acionáveis em produção hoje — e não existe um único teste automatizado, backup ou alerta de incidente. Com as correções P0 deste relatório (estimadas em 1 a 2 semanas de trabalho), o sistema passa a suportar crescimento de 10x com segurança razoável.*

O Simulados Policiais Pro é uma SPA Next.js executada quase inteiramente no aparelho do aluno: o PDF nunca sai do dispositivo, a extração de texto acontece no navegador (unpdf) e todo o progresso de estudo (documentos, tentativas, caderno de erros) vive em IndexedDB. Essa decisão é o maior ativo arquitetural do projeto: o componente mais caro de escalar — o processamento por usuário — é, por construção, distribuído nos aparelhos dos próprios usuários. O servidor responde em 39 ms na homepage e não guarda dados de estudo.

O problema está no que sobrou no servidor. O "banco de dados" de contas e assinaturas é uma coleção de arquivos JSON no Vercel Blob, varrida sequencialmente com teto de 2.000 usuários (src/lib/blob-registry.ts:112,139-147), com índices de unicidade reescritos integralmente sem lock (src/lib/user-auth.ts:205-256) e com um heartbeat que reescreve o JSON completo do usuário a cada 5 minutos (src/app/api/auth/me/route.ts:41). Em paralelo, o webhook de pagamento aceita operar sem segredo configurado, sem HMAC e sem idempotência (src/app/api/webhook/cakto/route.ts:47-54, 56-110), o painel administrativo devolve o hash de senha de todos os usuários na resposta HTTP (src/app/api/admin/overview/route.ts:47-53,103) e as rotas de geração por IA estão abertas sem sessão nem rate limit — confirmado por probe em produção, que respondeu 400 (validação de conteúdo) em vez de 401 para uma requisição anônima.

3,2/10	7	0	2.000
nota geral de prontidão (100k)	riscos críticos com evidência	testes automatizados / CI	teto de usuários visível no admin

Os sete riscos críticos (P0), em ordem de urgência: **(1)** token da API da Vercel hardcoded em arquivo commitado, a um push do GitHub; **(2)** webhook da Cakto que vira assinatura paga sem validação criptográfica e aceita replay; **(3)** exposição do hash de senha e PII completa via /api/admin/overview; **(4)** proxy de IA aberto e sem limite custeado pelo dono do app; **(5)** registro central em JSON sem banco, sem índice e sem lock; **(6)** heartbeat com amplificação de escrita que, a 100 mil usuários, chega a aproximadamente 1,2 milhão de escritas por hora; **(7)** ausência total de conformidade LGPD — o titular não consegue excluir nem exportar os próprios dados. Nenhum deles exige reescrever o produto; todos exigem decisão imediata.

2. Nota da Engenharia Atual

As notas abaixo avaliam prontidão para 100.000 usuários ativos — não a funcionalidade atual, que está entregue e operando em produção. O critério é: "este pilar suporta crescimento de 10x a 100x sem reconstrução de emergência?". Cada nota está ancorada nos achados das seções seguintes, com arquivo e linha; quando a evidência é um probe de produção, isso está indicado.

Pilar	Nota	Justificativa em uma linha
Arquitetura	6,0	Local-first bem executado no cliente; servidor sem camada de dados real (JSON no Blob).
Segurança	2,5	Hash scrypt e sessão HMAC corretos, mas 7 vetores críticos abertos (webhook, overview, IA, secrets).
Banco de dados	2,0	Não há banco server: JSON no Blob com varredura O(N), cap 2.000, unicidade sem lock, zero backup.
Escalabilidade	3,5	Cliente escala por construção; o registro central colapsa entre 10k e 50k contas.
Performance	5,0	TTFB 39 ms (probe); riscos estruturais: bundle 384 KB de dados, PDF na main thread, O(N) no admin.
Resiliência	3,0	Timeouts e fallback de IA bem feitos; sem retries/backoff, circuit breaker, idempotência ou DR.
Observabilidade	1,5	6 console.* sem estrutura, sem request-id, sem error tracking; PII em log de webhook.
Testes	0,5	Zero testes automatizados; apenas scripts manuais em tests/; sem CI.
Manutenibilidade	5,0	Código legível e comentado, mas telas de 800-1.354 linhas, TS/ESLint desligados, ~18 deps mortas.
DevOps/Deploy	3,0	Vercel + GitHub integrados; deploy manual, sem CI, secrets pendentes de rotação, .env commitado.

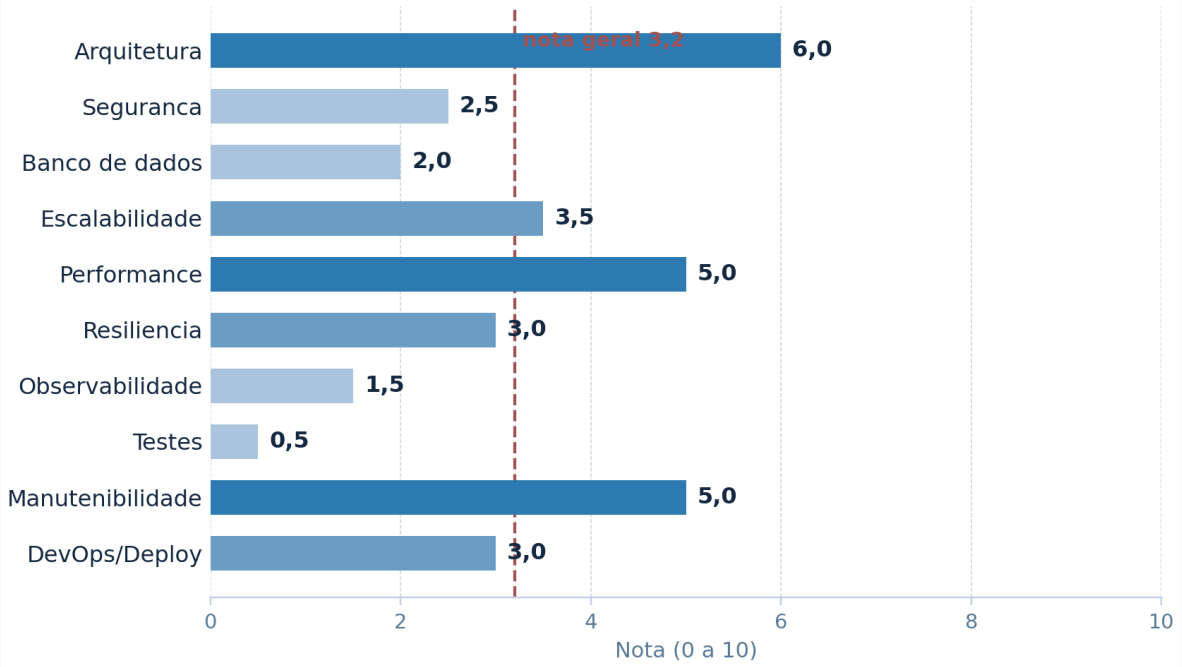


Figura 1 — Notas por pilar (0-10). Linha tracejada: nota geral 3,2.

NOTA GERAL: 3,2/10. A média simples dos dez pilares é 3,25. A leitura correta desse número é assimétrica: os pilares altos (arquitetura 6,0, performance 5,0, manutenibilidade 5,0) mostram que o design de produto é competentemente executado; os pilares baixos (testes 0,5, observabilidade 1,5, banco 2,0, segurança 2,5) são exatamente os que falham primeiro e mais caro sob crescimento. Não é uma nota de qualidade de software escrita — é uma nota de risco operacional a 100k usuários.

3. Arquitetura Atual

O sistema real funciona assim: o aluno abre a SPA (uma única rota /, treze telas trocadas por useState, sem deep-links); o PDF selecionado é processado no próprio aparelho pelo unpdf/pdf.js e fatiado em blocos de até 7.000 caracteres (máximo 10 blocos por documento); cada bloco vai por POST a /api/generate com a chave de IA do aluno no corpo (BYOK), e as questões geradas voltam para o IndexedDB. Conta, trial de 7 dias, assinatura Cakto e perfil (nome, foto em data URL de até 160 KB) vivem no servidor — porém não em um banco: em arquivos JSON no Vercel Blob, um por usuário, manipulados por src/lib/blob-registry.ts via read-modify-write. O painel /admin (cookie próprio spp_admin, HMAC 12h) lê todos os JSONs para calcular métricas. Pagamentos entram por webhook da Cakto ou ativação consultando a API da Cakto em tempo real.

Tecnologia	Onde aparece	Função	Risco principal
Next.js 16 App Router, React 19	src/app (68 "use client", 2 server)	SPA de 13 telas + 20 rotas API	Tudo client-renderizado; sem deep-links; erro de TS ignorado no build.
Tailwind CSS 4	globals.css	Tema claro/escuro próprio (classe .dark)	Baixo; overrides manuais exigem disciplina.
IndexedDB (API crua, v2)	src/lib/local-db.ts	documents + attempts + errors	Sem migração de schema; QuotaExceededError engolido em silêncio.
localStorage (11 chaves)	src/lib/license.ts, ai-settings.ts etc.	trial, deviceUid, chave de IA, metas	Chave de IA em texto plano; deviceUid controlado pelo cliente.
unpdf / pdf.js	src/lib/pdf-text.ts, pdf-ocr.ts	Extração de PDF no aparelho	Main thread; pico de RAM com PDFs de 50 MB em iOS.
Vercel Serverless (Node)	src/app/api/** (20 rotas)	Auth, licenças, IA, admin, webhook	maxDuration até 150s; estado em memória por instância.
Vercel Blob (JSON)	src/lib/blob-registry.ts	Contas, índices, eventos, códigos	Não é banco: O(N), cap 2.000, read-modify-write sem lock.
Google Gemini (BYOK + nativo)	src/lib/ai-gateway.ts	Geração de questões e OCR	Chave transita pelo servidor (não é persistida); fallback de modelos bem implementado.
Z.ai SDK + gateway OpenAI-compat	generate, transcribe, ai/test	Provedores custeados pelo app	SDK sem timeout; expostos sem rate limit (achado 5).
Cakto (checkout + API)	src/config/site.ts, src/lib/cakto.ts	Assinatura R$ 17,90/mês	API sem timeout; webhook com segredo opcional.

Tecnologia	Onde aparece	Função	Risco principal
GitHub + Vercel	repo clodoaldosilva608/simulados-policiais-pro	Deploy por push	Sem CI/testes; token Vercel commitado (achado 1).
Vercel Edge (headers)	probe de produção	TLS + HSTS padrão	Sem CSP, X-Frame-Options, X-Content-Type-Options.

FATO OBSERVADO estrutural: o Prisma existe no package.json e em prisma/schema.prisma (provider sqlite), mas não há um único import de @prisma/client em src/ — o schema é um fóssil de uma fase anterior (e incompatível com serverless: SQLite exige filesystem local). Da mesma forma, next-auth está instalado e não é usado. A autenticação real é artesanal e correta no essencial: scrypt + salt + timingSafeEqual para senhas (src/lib/user-auth.ts:41-58) e token HMAC-SHA256 em cookie httpOnly/Secure/SameSite=Lax de 30 dias (:71-122).

4. Diagrama do Sistema

O diagrama abaixo reconstrói o fluxo real — usuário, aparelho, serverless, persistência e serviços externos — a partir do código, não da intenção de design. As setas vermelhas marcam os dois fluxos críticos confirmados: a geração de IA custeada pelo app (aberta a anônimos) e o webhook de pagamento forjável/replayável.

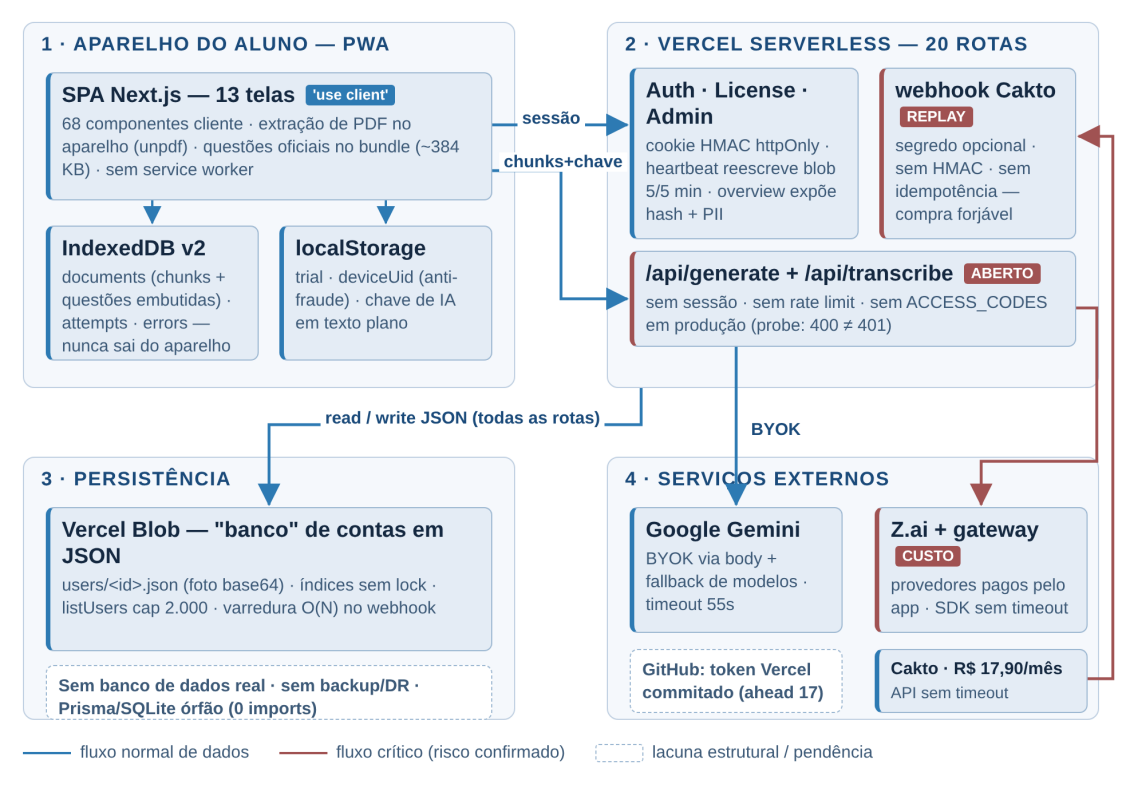


Figura 2 — Arquitetura real do Simulados Policiais Pro (reconstruída do código). Vermelho: fluxos críticos. Tracejado: lacunas estruturais.

Três leituras importantes do diagrama. Primeira: nenhuma seta sai do aparelho levando dado de estudo — o conteúdo de estudo nunca deixa o dispositivo, o que torna o isolamento entre usuários de estudo trivialmente correto (seção 8). Segunda: todas as rotas do serverless convergem para o mesmo ponto único — o Vercel Blob de JSONs — que é ao mesmo tempo banco, índice e fila de eventos; qualquer gargalo ali é gargalo de tudo (seções 9, 10 e 15). Terceira: a chave BYOK do aluno entra no corpo do POST e atravessa a função serverless antes de chegar ao Google; ela não é persistida (grep confirma zero escritas), mas transita pela plataforma — correto, porém exige disciplina permanente de não logar o corpo da requisição.

5. Top 20 Problemas

Consolidação dos achados verificados, ordenada por severidade e prioridade. "Evidência" cita arquivo e linha; probes de produção estão marcados. P0 = corrigir imediatamente; P1 = antes de crescer; P2 = manutenção programada.

#	Severidade	Problema	Evidência	Impacto	Prio.
1	CRÍTICO	Token da API Vercel (vcp_) hardcoded em arquivo commitado; branch a 17 commits do push	scripts/set-vercel-ai-env.py:5; git ls-files; git status (ahead 17)	Comprometimento total da infra de deploy	P0
2	CRÍTICO	Webhook Cakto fica totalmente aberto se CAKTO_WEBHOOK_SECRET não estiver configurada	api/webhook/cakto/route.ts:47-54	Assinatura "pro" forjável por POST anônimo	P0
3	CRÍTICO	Webhook sem HMAC e sem idempotência; replay de purchase_approved reativa assinatura cancelada	route.ts:56-110 (upsert em :67-80)	Receita contornável; estados inconsistentes	P0
4	CRÍTICO	/api/admin/overview retorna passwordHash, foto base64, notas e códigos da env de todos os usuários	admin/overview/route.ts:47-53,103,113-116	Vazamento de credenciais offline-quebráveis e PII	P0
5	CRÍTICO	/api/generate e /api/transcribe sem sessão nem rate limit; ACCESS_CODES ausente em produção (probe: 400, não 401)	generate/route.ts:21-31,136 + probe	Proxy de IA aberto custeado pelo dono	P0
6	CRÍTICO	Registro central em JSON no Blob: listUsers cap 2.000, leitura sequencial, findUserByEmail O(N)	blob-registry.ts:112,139-154	Latência/custo explosivos; truncamento silencioso	P0
7	CRÍTICO	Heartbeat reescreve o JSON completo do usuário a cada 5 min, sem lock (corrida com webhook/admin)	auth/me/route.ts:41; blob-registry.ts:168-218	~1,2M escritas/h a 100k; perda de status de assinatura	P0
8	CRÍTICO	LGPD: nenhum endpoint de exclusão ou exportação pelo titular (busca sem resultados)	src/app/api (grep "delete")	Risco regulatório direto (LGPD art. 18)	P0
9	ALTO	Anti-fraude do trial baseado em deviceUid gerado e guardado no localStorage	src/lib/license.ts:32-47; register/route.ts:49	Trial de 7 dias renovável indefinidamente	P1
10	ALTO	Questões oficiais (~384 KB de TS) embutidas no bundle do cliente; gate de telas é cosmético	src/data/*.ts; page.tsx:19	Conteúdo pago extraível; TTI degradado	P1

#	Severidade	Problema	Evidência	Impacto	Prio.
11	ALTO	Sem backup/DR de contas e assinaturas (busca sem resultados em scripts/docs)	repo (grep backup/restore)	Perda de MRR e de contas em incidente único	P1
12	ALTO	Segredo do webhook em query string e comparação com !== (não timing-safe)	webhook/cakto/route.ts:11-14,51	Segredo vaza em logs de acesso/CDN	P1
13	ALTO	Sem recuperação de senha e sem verificação de e-mail (busca sem resultados)	src/lib/user-auth.ts (grep reset/verify)	Contas irrecuperáveis; cadastro com e-mail alheio	P1
14	ALTO	Rate limits em Map na memória da instância serverless	user-auth.ts:379-390; admin-auth.ts:125	Brute force multiplicado pelo número de instâncias	P1
15	ALTO	Sem CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy (probe confirma)	next.config.ts:1-12 + probe de headers	Clickjacking, sniffing, XSS amplificado	P1
16	ALTO	Delete admin permanente sem soft-delete nem trilha de auditoria de quem agiu	admin/users/route.ts:123-133	Destruição em massa com um cookie roubado	P1
17	ALTO	Sessões aceitas também via header Authorization: Bearer (token portador, sem escopo)	user-auth.ts:104-111; admin-auth.ts:97-104	Token em log de proxy reutiliza sessão de 30d	P1
18	MÉDIO	.env e worklog.md commitados; PAT do GitHub embutido na URL do remote	git ls-files; git remote -v	Padrão que garante vazamento futuro de secrets	P2
19	MÉDIO	ignoreBuildErrors:true + ESLint quase desligado + noImplicitAny:false	next.config.ts:6-8; eslint.config.mjs	Bugs de tipo e regressões passam silenciosas	P2
20	MÉDIO	Zero testes automatizados, zero CI, zero error tracking (Sentry etc.)	repo (grep test/spec/jest)	Regressões invisíveis; incidentes sem alerta	P2

6. Buracos Encontrados

Além dos problemas pontuais, a varredura revelou lacunas estruturais — coisas que simplesmente não existem e cuja ausência cobra preço assim que o sistema cresce. Elas estão listadas sem diplomacia, cada uma com a consequência concreta de continuar assim.

- **Autenticação incompleta:** não existe esqueci-minha-senha nem verificação de e-mail. Quem perde a senha depende de suporte manual; quem cadastra e-mail de terceiro trava esse e-mail para sempre (user-auth.ts:281).
- **LGPD inexistente:** sem exclusão de conta pelo titular, sem exportação de dados, sem política de retenção. A única deleção é administrativa e apaga tudo de uma vez (admin/users/route.ts:123-133).
- **Nenhum backup:** o "banco" inteiro (contas, assinaturas, códigos) são JSONs no Blob sem cópia de segurança, sem dump, sem restore documentado. Perda = perda de receita.
- **Nenhuma observabilidade:** sem Sentry, sem métricas, sem alertas, sem request-id; 6 chamadas console.* em todo o servidor, uma delas logando PII de comprador (webhook/cakto/route.ts:60).
- **Nenhum teste e nenhum CI:** zero arquivos *.test.*/*.spec.*, zero workflow .github, zero vercel.json. Todo deploy é uma aposta.

- **Schema fóssil:** prisma/schema.prisma (sqlite, sem modelo User) e next-auth instalados e mortos; scripts db:* com --accept-data-loss prontos para engano.
- **Anti-fraude cosmético:** o trial "1 por aparelho" depende de um deviceUid gerado no localStorage (license.ts:32-47) — limpar o storage cria novo trial; e o conteúdo pago (questões oficiais) viaja no bundle.
- **Sem service worker:** o app é "local-first" mas não funciona offline de forma declarada; o manifesto PWA existe, cache nenhum.
- **Sem documentação:** README e docs/ não existem; a única memória é o worklog.md — que está commitado e documenta segredos (item 18 do Top 20).
- **Código monolítico no cliente:** library-screen.tsx com 1.354 linhas acumulando header, menu, biblioteca e dialogs; zero next/dynamic em toda a app.
- **~18 dependências mortas:** next-auth, next-intl, react-query, react-table, mdxeditor, dnd-kit, zustand, zod, uuid, sharp e componentes ui/ órfãos — superfície de ataque e custo de build gratuitos.
- **Filas inexistentes:** geração de IA (maxDuration 150) e sincronização Cakto (maxDuration 60) rodam dentro da requisição HTTP; a 10k+ isso vira timeout e custo.

7. Segurança

A auditoria de segurança começou pelos pontos que estão certos — e eles existem: senhas com scrypt, salt aleatório e comparação timing-safe (user-auth.ts:41-58); sessão HMAC-SHA256 em cookie httpOnly/Secure/SameSite=Lax (:113-122); status de banimento checado no servidor a cada request (:162-174); admin falha fechada quando ADMIN_PASSWORD_HASH não está configurada (admin-auth.ts:34-40); probe em produção confirma 401 correto no overview sem sessão. O problema não é o que foi escrito; é o que falta ao redor.

7.1 Autenticação e sessão

Achado	Severidade	Evidência	Por que é perigoso
Segredo da sessão do aluno com fallback para o segredo do admin	MÉDIO	user-auth.ts:72	Rotacionar o segredo do painel invalida/afeta todas as sessões de alunos; domínios de confiança acoplados.
Tokens aceitos via Bearer (portador)	ALTO	user-auth.ts:104-111	Token vazado em log/proxy reutiliza sessão de 30 dias sem as proteções do cookie.
Troca de senha não invalida sessões (sem jti/rotação)	MÉDIO	user-auth.ts:64-84; password/route.ts	Conta comprometida continua acessível pelo atacante por até 30 dias.
Sem reset de senha e sem verificação de e-mail	ALTO	busca sem resultados	Recuperação manual via suporte; cadastro com e-mail de terceiro trava o e-mail.
Sem 2FA no admin; brute-force limitado por instância	MÉDIO	admin-auth.ts:125-141	Limite em memória é multiplicado pelas instâncias serverless.

7.2 Webhook de pagamento (pior área do projeto)

O webhook da Cakto é o caminho que transforma dinheiro em assinatura — e é o código com menor resistência a adversário. A validação é opcional: se CAKTO_WEBHOOK_SECRET não existir, nenhuma checagem é feita (route.ts:47-54); quando existe, viaja em query string (?secret=...) e é comparado com !== (:49-51), não de forma timing-safe. Não há assinatura HMAC do corpo, não há id de evento, não há timestamp, não há deduplicação: um replay de purchase_approved depois de um refund reativa a assinatura (:67-80), e o ramo de cancelamento cria usuários novos com e-mail arbitrário do payload (:81-95). O corpo da requisição, com PII do comprador, é logado em texto (:60). Em produção, com o segredo configurado, o risco residual é vazamento do segredo por logs + replay; sem o segredo, o risco é assinatura gratuita ilimitada.

7.3 Painel administrativo

- **Exposição de credenciais (CRÍTICO):** o GET /api/admin/overview espalha o objeto completo de cada usuário — incluindo passwordHash scrypt, foto base64, notas administrativas — na resposta JSON (admin/overview/route.ts:47-53,103); hash de senha com mínimo de 6 caracteres (user-auth.ts:276) é offline-quebrável.
- **Códigos de acesso no corpo da resposta (ALTO):** accessCodesEnv devolve os códigos da env ACCESS_CODES (:113-116); vazamento por extensão/print/log permite ativação fraudulenta em /api/license.
- **Delete permanente (ALTO):** action=delete apaga a conta sem soft-delete, sem confirmação adicional e sem registrar quem agiu (admin/users/route.ts:123-133).
- **Materialização total (ALTO):** o overview lê até 2.000 JSONs por request e devolve todos; a milhares de contas, a resposta passa de dezenas de MB e as métricas ficam truncadas sem aviso (blob-registry.ts:112).

7.4 Superfície de IA e headers

As rotas /api/generate, /api/transcribe e /api/ai/test não exigem sessão; o único gate é o header estático x-access-code contra a env ACCESS_CODES — e quando a env não existe, hasValidAccess() retorna true (generate/route.ts:21-31). O probe de produção (POST com corpo vazio) respondeu 400 "Bloco de texto muito curto" — validação de conteúdo, que roda DEPOIS da checagem de acesso (:136 antes de :151) — provando que o gate está inativo em produção: qualquer anônimo com texto real consome os provedores custeados pelo app (cadeia zai → gateway-do-app, :171-172). /api/ai/test, sem rate limit, funciona como validador anônimo de chaves Gemini. Nos headers, o probe confirmou apenas HSTS (padrão Vercel); CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy e Permissions-Policy não existem (next.config.ts tem apenas output standalone e ignoreBuildErrors).

8. Multi-Tenancy e Isolamento

O tenant é a conta individual (uid). A identificação chega ao backend pelo cookie spp_user (HMAC de {uid, deviceUid, exp}, httpOnly) e a cada request o servidor revalida status/banimento contra o registro central (user-auth.ts:162-174) — correto: o frontend não é camada de segurança para o status de acesso. A pergunta central da auditoria — "o usuário A consegue acessar, descobrir, alterar ou excluir dados do usuário B?" — tem resposta em duas partes distintas.

Dados de estudo: não. Documentos, tentativas, respostas e caderno de erros nunca saem do aparelho (IndexedDB); o servidor não possui endpoint que leia conteúdo de estudo de qualquer usuário — nem o admin acessa respostas de simulados, apenas metadados de conta. O isolamento aqui é por construção física, não por filtro de query, e por isso resiste a bugs de aplicação.

Dados de conta/assinatura: parcialmente. Três vetores quebram o isolamento no plano server: **(a)** /api/license ativa a conta logada informando apenas o e-mail declarado da compra (license/route.ts:103-126) — quem comprou pode ativar a conta de outrem, e uma assinatura pode alimentar várias contas; **(b)** /api/registry/track aceita um uid vindo do cliente e cria blob de usuário (registry/track/route.ts:55-70) — poluição e enumeração possíveis; **(c)** o overview admin expõe PII e hash de todos (achado 4). Além disso, o deviceUid do anti-fraude é escolhido pelo cliente (license.ts:32-47), o que esvazia a garantia "1 trial por aparelho".

9. Banco de Dados e Camada de Dados

Do lado do servidor não existe banco de dados. O prisma/schema.prisma declara provider sqlite (linhas 5-8) com modelos Document/Question/TextBlock/Attempt, sem modelo User e sem nenhuma @@unique; não há pasta de migrations. Nenhum arquivo de src/ importa @prisma/client (grep: zero resultados) — e é impossível usar esse schema em Vercel serverless (filesystem efêmero), o que explica o abandono. O que opera de verdade é src/lib/blob-registry.ts: um arquivo JSON por usuário no Vercel Blob (users/.json, com foto base64 embutida), mais índices em JSON (index/auth-emails.json, index/device-trials.json), eventos (cap 250) e configurações.

Mecanismo	Código	Comportamento real	Consequência em escala
listUsers()	blob-registry.ts:139-147 (cap 2000 em :112)	lista caminhos e lê um JSON por usuário, sequencialmente	painel e métricas truncados em 2.000 contas; latência e egress crescem linearmente
findUserByEmail()	:149-154	varredura completa O(N) — usada no webhook e no login legado	cada compra dispara scan; O(S × U) no sync Cakto
claimEmail / markDeviceTrial	user-auth.ts:205-211, 251-256	lê o índice JSON inteiro, muta, reescreve integralmente, sem lock	lost update: e-mail/trial duplicados sob concorrência; índice de 100k contas reescrito a cada signup
upsertUser()	blob-registry.ts:168-218	read-modify-write sem CAS/versão	heartbeat + webhook + admin intercalados perdem campos (ex.: banimento sobrescrito)

Mecanismo	Código	Comportamento real	Consequência em escala
appendEvent()	:238-259	para cada evento, reler todos e podar acima de 250	cada append custa centenas de reads; auditoria de compras é descartada
heartbeat	auth/me/route.ts:41	GET do cliente dispara upsertUser completo (incl. foto)	a 100k MAU: ~1,2M escritas Blob/hora; corrida com webhook

Do lado do cliente, o IndexedDB (db simulados-policiais-pro, v2, API crua) guarda documentos com chunks do PDF, questões, textBlocks e ocrText embutidos no MESMO registro (local-db.ts:20-43) — reescrito integralmente a cada bloco processado (upload-dialog.tsx:170-177), degradando de forma quadrática em PDFs grandes. Não existe tratamento de QuotaExceededError: os catch engolem o erro e o usuário perde o simulado em silêncio (local-db.ts:199,335,404). Tentativas crescem sem limite e sem virtualização na UI. **Backups: NÃO FOI POSSÍVEL DETERMINAR qualquer mecanismo** — a busca por backup/restore/snapshot/dump em scripts, docs e código retornou zero resultados aplicáveis.

10. Escalabilidade (1.000, 10.000, 50.000, 100.000)

A análise por patamar considera usuários ATIVOS no mês (MAU) e o comportamento real do código, não especulação genérica. O cliente aguenta todos os patamares — o servidor, não.

Patamar	O que acontece	Evidência
1.000	Operação normal. Homepage 39 ms (probe). Riscos dominantes são os de segurança (seção 7), não de carga. Heartbeat: ~14 mil escritas Blob/dia — tolerável.	probe; blob-registry.ts
10.000	Rate limits em memória perdem eficácia (× instâncias); overview admin começa a ler milhares de JSONs por request; custo de Blob (requests + egress) cresce visível na fatura; heartbeat ~140 mil escritas/dia.	user-auth.ts:379-390; admin/overview/route.ts:46
50.000	findUserByEmail O(N) no caminho do webhook e do login legado: cada compra varre dezenas de milhares de JSONs; cap de 2.000 trunca o painel silenciosamente (métricas erradas); corrida heartbeat × webhook começa a perder campos; egress de ~160 KB/foto por overview.	blob-registry.ts:112,149-154; auth/me:41
100.000	Colapso do registro central: unicidade sem lock duplica contas/trials; upsert read-modify-write perde status de assinatura sob concorrência; Blob rate-limited por ~1,2M escritas/h; admin inutilizável; custo de egress dominante. O app cliente continua funcionando — o que quebra é conta, assinatura e painel.	blob-registry.ts:168-218; auth/me:41

Conclusão do pilar: o desenho local-first empurrou 95% da carga para o aparelho do usuário — excelente decisão de escala. O 5% restante (contas, assinaturas, webhooks) concentra 100% do risco, porque foi construído sobre um storage de objetos usado como banco transacional. A correção não é mais servidor nem cache: é um banco de verdade para o registro central (seção 17).

11. Performance

11.1 Frontend

- **Bundle:** zero next/dynamic/React.lazy em toda a app — as 13 telas + framer-motion (importado em 11 telas) + ~384 KB de questões oficiais (src/data/, maior arquivo 80 KB) entram no pacote inicial. Não medido em gzip (sem build no modo leitura); estruturalmente, o TTI degrada a cada questão adicionada ao catálogo.
- **Pipeline de PDF na main thread:** unpdf roda sem Web Worker (pdf-text.ts:72); um PDF de 50 MB vira ArrayBuffer + cópia Uint8Array + estruturas do pdf.js simultaneamente — risco real de crash no Safari iOS em aparelhos modestos. No caminho OCR, o buffer é lido duas vezes (:369, :407).
- **Listas sem virtualização:** o histórico renderiza filteredAttempts.map() completo (performance-screen.tsx:524) com allAttempts inteiro em state (page.tsx:266,316-317) — usuários intensos acumulam milhares de registros.
- **Re-renderização:** todo o app vive no orquestrador page.tsx (792 linhas); cada resposta no simulado re-renderiza o orquestrador e serializa o estado para localStorage (:291-298). Tolerável hoje; gargalo estrutural amanhã.
- **Sem service worker:** manifesto PWA presente, cache offline inexistente — o "local-first" não sobrevive a modo avião.

11.2 Backend

- **Funções longas:** /api/generate e /api/transcribe com maxDuration 150 — processamento de IA dentro da request; a 10k+ usuários simultâneos, isso consome concorrência da plataforma e multiplica custo.
- **N+1 no admin:** o overview dispara até 2.000 leituras de Blob por request (blob-registry.ts:139-147) e devolve tudo no corpo.
- **Heartbeat caro:** cada GET /api/auth/me reescreve o JSON do usuário incluindo foto base64 de até 160 KB (auth/me/route.ts:41) — tráfego e escrita amortizados a cada 5 min por usuário ativo.
- **O que já é bom:** timeouts de 55/58 s com AbortController no gateway de IA (ai-gateway.ts:90-99,367,391) e cadeia de fallback de modelos com deadline global de 130 s (:170-205,272) — o melhor trecho de engenharia do servidor.

O que degrada primeiro em performance: com 10k MAU, o painel admin e o egress do Blob; com 50k, as funções de IA sob concorrência e o scan O(N) do webhook; com 100k, o heartbeat e o próprio Blob. A experiência do aluno final — a parte que gera receita — permanece estável porque roda no aparelho.

12. Resiliência

A matriz abaixo simula a falha de cada dependência e compara a proteção existente com a lacuna. Padrão geral: o gateway de IA está bem protegido; todo o resto reage por acidente, não por design.

Se falhar...	O que acontece hoje	Proteção existente	Lacuna
Vercel Blob fica lento/fora	login/registro/webhook falham; heartbeat engolido por catch vazio	try/catch pontual (auth/me:42-44)	sem fallback, sem fila, sem alerta; lastSeen e eventos perdidos
Gemini fica lento	geração pendurada até o limite	AbortController 55/58s + cadeia de 3 modelos + retry (ai-gateway.ts:170-205)	sem circuit breaker (requisições seguem para provedor doente)
Z.ai SDK fica lento	request pendurada	2 tentativas na rota (generate:96-113)	sem timeout explícito no SDK
Cakto API fica lenta	/api/license presa até maxDuration	limites de plataforma (60s/30s)	sem timeout no cliente HTTP (cakto.ts:22-27,82-85)
Webhook chega 2x (retry da Cakto)	usuário duplicado / assinatura reativada	nenhum	sem idempotência nem dedupe de evento
Instância serverless reinicia	rate maps zerados; limites reiniciam	nenhum	limites efetivos multiplicados por N instâncias
Usuário recarrega no meio da geração	chunk re-enviável sem perder progresso	geração resumível por chunk (upload-dialog:148-181)	sem dedupe de custo (o chunk re-pago é invisível)

13. Observabilidade

Resposta direta às duas perguntas do roteiro. **"Se o sistema parar para 5.000 usuários às 3h da manhã, saberíamos?" Não.** Não existe error tracking (Sentry/Datadog: zero resultados em package.json e src/), não existe métrica, não existe alerta, não existe health check consumido. **"Saberíamos qual componente causou?" Também não.** São 6 chamadas console.* em todo o servidor, sem log estruturado, sem request-id, sem correlation-id; a única linha que loga payload (webhook/cakto/route.ts:60) registra PII do comprador em texto plano — o pior dos dois mundos: pouca visibilidade e vazamento onde existe.

- **O que existe:** logs de plataforma da Vercel (duração/erros de função), 6 console.*, trilha de eventos no Blob (appendEvent cap 250).
- **O que falta para operar profissionalmente:** error tracking com source maps; log estruturado JSON com request-id; métricas por rota (p95, taxa de erro, custo por chamada de IA); alerta em falha de webhook e em erro 5xx sustentado; trilha de auditoria de ações admin (quem, quando, o que).
- **Risco de negócio associado:** fraude no webhook e abuso de IA são invisíveis até aparecerem na fatura ou na reclamação de usuário — hoje não há como detectar nem delimitar um incidente.

14. Custos

NÃO FOI POSSÍVEL DETERMINAR o custo absoluto — sem acesso ao plano da Vercel, às métricas de uso atuais e aos volumes de IA consumidos, qualquer número seria inventado. O que a auditoria identifica são os vetores de explosão de custo, todos ligados a achados já citados, e as métricas que precisam começar a ser coletadas agora para viabilizar estimativa futura.

- **IA custeada pelo app:** com /api/generate e /api/transcribe abertos (achado 5), cada request anônimo custa dinheiro real (zai → gateway). Este é o único vetor capaz de gerar prejuízo em dias, não meses.
- **Escritas e egress do Blob:** heartbeat + appendEvent + overview multiplicam requests de escrita/leitura; fotos base64 de 160 KB dentro dos JSONs inflacionam o egress a cada leitura admin.
- **Funções de 150 s:** concorrência e duração de /api/generate consomem o limite de execução do plano e viram timeout pago sem retorno.
- **Logs:** sem retenção/política, o volume cresce sem controle (e hoje já carrega PII).

Métricas a coletar (pré-requisito de qualquer estimativa): MAU; chamadas/dia em /api/generate e /api/transcribe (por chave BYOK vs custeada); tokens médios por geração; GB-mês e operações do Blob por prefixo; duração p95 das funções; egress do overview admin. Com 30 dias dessas séries, a estimativa de custo por usuário ativo sai direto dos dados.

15. O Que Quebra Primeiro

Previsão técnica baseada exclusivamente na arquitetura encontrada — sem especulação genérica.

1. **1º — O registro central (Blob JSON).** É o único componente no caminho de TODAS as operações com conta. Com dezenas de milhares de contas: findUserByEmail O(N) no webhook estoura maxDuration (compras deixam de ativar assinatura), unicidade sem lock duplica registros, o cap de 2.000 esconde usuários do admin. É aqui que o negócio para de funcionar antes de o site cair.
2. **2º — O custo e os provedores de IA.** Rotas abertas sem sessão nem rate limit (confirmado em produção): abuso automatizado converte o produto em proxy gratuito e a fatura de IA em passivo. Pode preceder o item 1 se o tráfego crescer mais rápido que a base de contas.
3. **3º — O limite de escrita do Blob via heartbeat.** ~1,2M escritas/hora a 100k MAU aciona rate limiting da plataforma; efeito colateral pior: a corrida heartbeat × webhook × admin com read-modify-write sem lock perde status de assinatura — alunos pagantes bloqueados sem motivo aparente.
4. **4º — O painel administrativo.** O overview materializa até 2.000 JSONs + fotos base64 por request: dezenas de MB, timeout, métricas truncadas — o dono perde visibilidade exatamente quando mais precisa dela.
5. **5º — A operação/suporte.** Sem observabilidade, sem reset de senha, sem LGPD, sem testes: cada incidente vira crise manual e cada reclamação vira ticket de engenharia. Não derruba o sistema;

derruba quem o opera.

16. Roadmap de Escala

AGORA (esta semana — P0)

- Revogar e rotacionar os três segredos expostos (Vercel vcp_, GitHub ghp_, Cakto client_secret); remover scripts/set-vercel-ai-env.py do git (git rm --cached) e purgar .env/worklog do rastreamento.
- Webhook Cakto fail-closed: exigir segredo sempre + idempotência mínima por (email, evento, timestamp) + remover o log do corpo.
- Projetar AdminUserView explícita: overview SEM passwordHash, SEM foto, SEM notes, SEM accessCodesEnv.
- Gate de sessão + status em /api/generate, /api/transcribe e /api/ai/test (validateUserSession + accessOf) + rate limit por uid.
- Heartbeat sem escrita (ou throttled para 1x/dia).

ANTES DE 1.000 USUÁRIOS

- Migrar o registro central para Postgres gerenciado (Neon/Supabase) com modelo User real, unique(email), upsert atômico e pooling (directUrl/pgbouncer).
- Backup diário automático do banco + teste de restore.
- Sentry + log estruturado com request-id; alerta em 5xx sustentado e falha de webhook.
- Reset de senha + verificação de e-mail; endpoints LGPD de exclusão e exportação.
- CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy; ignoreBuildErrors:false; ESLint mínimo ativo.

ANTES DE 10.000 USUÁRIOS

- Fila serverless (QStash/Inngest) para geração de IA + quota diária por usuário; tirar o processamento da request.
- Rate limit distribuído (Upstash Redis) substituindo os Maps em memória.
- CI (lint + typecheck + build) e primeira suíte de testes: autenticação, webhook (replay/forgery) e licença.
- vercel.json com maxDuration e regiões explícitos; dashboard de métricas de custo (métricas da seção 14).

ANTES DE 50.000 USUÁRIOS

- Read replica/cache para leituras do painel; paginação real no overview.
- Code-splitting (next/dynamic por tela), Web Worker para PDF, virtualização do histórico.
- Split de library-screen.tsx (1.354 linhas) e extração de módulos compartilhados; remoção das ~18 deps mortas.
- Revisão de custo por usuário ativo contra a série coletada desde 10k.

ANTES DE 100.000 USUÁRIOS

- DR drills com RTO/RPO definidos; SLOs formais e error budget.
- WAF/edge rules na Vercel; pentest externo focado em auth/webhook/licença.
- Multi-região e sharding apenas se as métricas de latência exigirem — não antes.
- Conformidade LGPD formal: política de retenção, responsável nomeado, registro de operações.

17. Arquitetura Recomendada

A arquitetura-alvo é a **mais simples capaz de suportar 100.000 usuários**, não a mais sofisticada. O monolito modular atual é mantido; o que muda é a troca do pseudo-banco (Blob JSON) por um banco transacional gerenciado, e a saída do processamento pesado da request. Tabela de decisão componente a componente:

Componente	Decisão	Quando	Justificativa
Monolito modular Next.js (atual)	MANTER	Já é	Suficiente até 100k se a camada de dados for corrigida; o acoplamento real está nos dados, não no código.
Postgres gerenciado (Neon/Supabase)	ADOTAR	AGORA	Substitui o Blob de contas: unique(email), upsert atômico, transações, backups nativos, pooling.
Vercel Blob	REDUZIR	AGORA	Manter apenas se surgir upload server-side; hoje não precisa guardar nada essencial.
Fila serverless p/ IA (QStash/Inngest)	ADOTAR	Antes de 10k	Tira o maxDuration 150 da request; permite retry, quota e dedupe de custo.
Redis (Upstash) p/ rate limit	ADOTAR	Antes de 10k	Limites efetivos em multi-instância; contador por uid/IP compartilhado.
Sentry + logs estruturados	ADOTAR	AGORA (1k)	Custo mínimo, e é a diferença entre incidente de 5 min e de 5 h.
Load balancer / Kubernetes	NÃO	n/a	A Vercel já entrega escala horizontal; K8s aqui é overengineering puro.
Microserviços	NÃO	n/a	Não há domínio que justifique; o acoplamento atual é de dados, e o Postgres resolve.
CDN/edge cache	JÁ INCLUIDO	n/a	Vercel Edge entrega hoje (probe: 39 ms); nada a construir.

Componente	Decisão	Quando	Justificativa
WAF + pentest	ADOTAR	Antes de 50k	Superfície de pagamento + IA justifica camada gerenciada de filtro.

Com essa mudança, os achados 6 e 7 (Blob O(N) e heartbeat) desaparecem por construção — upsert atômico com unique(email) substitui o lock manual; o heartbeat vira UPDATE de uma coluna last_seen, ou deixa de existir. Os achados 2-5 se tornam regras de aplicação sobre uma base confiável, e o backup/DR passa a ser um recurso do banco, não um projeto.

18. Overengineering a Evitar

O roteiro pede explicitamente o que NÃO construir. Esta lista é tão importante quanto o roadmap: cada item abaixo gastaria semanas que os riscos P0 exigem para si.

- **Microserviços:** o produto tem 1 domínio, 1 equipe e 20 rotas. Nenhuma fronteira de negócio justifica rede interna, tracing distribuído ou contratos entre serviços.
- **Kubernetes/self-hosting:** a Vercel já resolve escala, TLS, edge e rollback. Migrar só aumentaria superfície operacional sem ganho de escala visível até 100k.
- **Arquitetura event-driven completa:** a fila recomendada (seção 17) é pontual — para IA. Transformar todo o fluxo em eventos agora seria complexidade sem demanda.
- **Cache distribuído antes de 10k:** com o Postgres no lugar, as leituras quentes (sessão/status) são queries de chave primária; o Redis entra quando as métricas mostrarem, não antes.
- **Multi-região:** usuários brasileiros, app client-heavy; a latência percebida é dominada pelo aparelho. Só com métricas provando necessidade.
- **GraphQL/CQRS/Kafka:** zero indicação nos dados atuais.
- **Reescrever o cliente:** a SPA local-first é o maior ativo do projeto — refatorar por estética (em vez de dividir page.tsx/library-screen com critério) seria queimar o diferencial.

19. Plano de Ação

FAZER IMEDIATAMENTE

1. Revogar/rotacionar Vercel vcp_, GitHub ghp_ e Cakto client_secret; git rm --cached dos arquivos com segredos e limpeza de histórico. (complexidade: baixa; horas)
2. Webhook Cakto fail-closed + idempotência por (email, evento, timestamp) + remover PII do log. (baixa-média; 1-2 dias)
3. Overview admin projetado (sem passwordHash/foto/notes/accessCodesEnv) + soft-delete e auditoria de ações admin. (baixa; 1 dia)

4. Sessão + status + rate limit em /api/generate, /api/transcribe, /api/ai/test. (média; 1-2 dias)
5. Heartbeat sem escrita de Blob (ou throttle 24h). (baixa; horas)

FAZER EM SEGUIDA

1. Migração do registro central para Postgres gerenciado (User real, unique email, upsert atômico) com cutover testado. (média-alta; 1 semana)
2. Backup diário + restore ensaiado. (baixa; 1 dia)
3. Sentry + log estruturado + alertas básicos. (baixa; 1-2 dias)
4. Reset de senha, verificação de e-mail e endpoints LGPD (excluir/exportar). (média; 3-4 dias)
5. Headers de segurança + ignoreBuildErrors:false + ESLint essencial + CI mínimo (lint, typecheck, build). (baixa; 1-2 dias)

PODE ESPERAR

1. Fila para IA + quota diária por usuário (antes de 10k).
2. Rate limit distribuído em Redis (antes de 10k).
3. Code-splitting, Web Worker de PDF, virtualização do histórico, split de telas gigantes.
4. Remoção das ~18 dependências mortas e do fóssil Prisma/next-auth.

NÃO MEXER AGORA

1. BYOK: o design (chave local, não persistida, testável) é válido — manter, apenas nunca logar o corpo.
2. Extração client-side de PDF: diferencial de custo e privacidade; evoluir com Web Worker, não substituir.
3. Monolito: não fragmentar em serviços; dividir arquivos grandes com critério, não com arquitetura.
4. Dark theme e cronômetro do trial: já corrigidos no commit 4efac13 (contraste das questões e unificação floor/ceil) — reabrir apenas se houver regressão.

20. Veredito Final

PERGUNTA 1 — O sistema está tecnicamente preparado para crescer 10x?

Não, na forma atual. 10x aciona simultaneamente os itens 1, 2, 3 e 5 do Top 20: o scan O(N) do webhook estoura maxDuration, o gate aberto de IA converte crescimento em prejuízo, o heartbeat amplifica escrita, e o painel perde visibilidade. Porém, este é o cenário de recuperação mais barato de toda a auditoria: com os cinco itens de "FAZER IMEDIATAMENTE" concluídos, 10x passa a ser suportável com folga — o cliente local-first faz o trabalho pesado.

PERGUNTA 2 — Está preparado para 100.000 usuários?

Não. Com modificações, sim. A condição necessária e suficiente é a migração do registro central para um banco transacional gerenciado (seção 17) mais o roadmap das seções 16 e 19. A arquitetura de cliente não precisa mudar; a de servidor precisa deixar de usar storage de objetos como banco. Nada aqui exige reconstruir o produto — exige corrigir a fundação da conta.

PERGUNTA 3 — Os 5 maiores riscos para o negócio

1. **Fraude de assinatura:** webhook forjável/replayável transforma receita recorrente em opcional para quem souber o endpoint.
2. **Custo de IA sem freio:** proxy aberto custeado pelo app — prejuízo em escala de dias, sem telemetria para detectar.
3. **Vazamento de hash de senha e PII via admin:** dano reputacional e exposição LGPD imediata.
4. **Perda de contas/assinaturas sem backup:** incidente único no Blob apaga a base de MRR sem caminho de volta.
5. **Segredos a um push do vazamento:** token Vercel commitado (branch a 17 commits do GitHub) pode comprometer toda a infraestrutura de deploy; e, no plano de produto, o trial infinito via deviceUid corrói a conversão.

PERGUNTA 4 — O menor conjunto de mudanças para reduzir drasticamente os riscos

Sete itens, todos da seção 19, em ordem: (1) rotacionar os três segredos e limpar o git; (2) webhook fail-closed com idempotência; (3) overview sem hash/PII/códigos; (4) sessão e rate limit nas rotas de IA; (5) heartbeat sem escrita; (6) backup diário com restore testado; (7) Sentry com alerta básico. Nenhuma dessas mudanças altera a experiência do usuário; juntas, eliminam quatro dos cinco riscos de negócio acima e tornam o quinto (trial) visível e gerenciável até a migração de banco.

Resposta final às duas perguntas do roteiro

"Se o sistema crescer 10x nos próximos meses, quais são os 5 problemas que mais podem colocar o negócio em risco?" Os cinco riscos da PERGUNTA 3, nesta ordem de probabilidade: custo de IA aberto, fraude de webhook, perda de status de assinatura por corrida de escrita, truncamento do painel (cegueira operacional) e vazamento de segredos. **"Qual é o menor conjunto de mudanças que devemos fazer AGORA?"** Os sete itens da PERGUNTA 4 — entre 1 e 2 semanas de trabalho concentrado, sem overengineering, sem reescrever nada que já funciona.